

Queue Mechanics

At the start of each adaptation round, a priority queue is assembled from all active elements. This determines the order in which the agent visits elements. The queue is a presentation ordering heuristic — it does NOT make decisions. The agent decides independently for every element, including elements the queue ranks as low priority.

Priority Computation

Each element’s priority is its distance from the neutral zone, measured in log-space:

$$\text{priority}(k) = \begin{cases} \log_{10}\left(\frac{e_k}{e_{\max}}\right) & \text{if } e_k > e_{\max} \quad (\text{under-refined}) \\ \log_{10}\left(\frac{e_{\min}}{e_k}\right) & \text{if } e_k < e_{\min} \quad (\text{over-refined}) \\ 0 & \text{if } e_{\min} \leq e_k \leq e_{\max} \quad (\text{neutral}) \end{cases} \quad (1)$$

Both under-refined and over-refined elements produce positive priorities. An element with error $10\times$ above $e_{\max}$ has priority 1; an element $100\times$ above has priority 2. The same scaling applies on the coarsening side. Neutral elements have priority 0.

The error indicators e_k are recomputed from the current mesh at the start of each round. The thresholds $e_{\max}$ and $e_{\min}$ remain fixed for the entire remesh interval ($\rightarrow$ Component 2, Steps 2–3).

Sorting and Presentation Order

Elements are sorted in descending order by priority. This means:

- **Highest-impact elements are presented first** — elements farthest from the neutral zone, whether they need refinement or coarsening.
- **No interleaving** between refine and coarsen candidates. Both produce positive priorities on the same scale, so the ordering is unified by urgency, not separated by action type.
- **Neutral elements sort to the end.** With priority 0, they are visited last. The agent still sees every neutral element and can act on it — but by the time it reaches them, the high-impact decisions have already been made.

Why this ordering matters: In a budget-constrained system, the order of decisions affects what resources remain for later decisions. By presenting the most urgent elements first, the agent makes its highest-impact choices while the full budget is still available. If the queue were random, the agent might exhaust the budget on low-priority elements before reaching the ones that matter most.

Why the queue is not a decision-maker: An earlier design (the “U-queue”) classified elements into refine and coarsen bins, effectively pre-deciding what each element needed. This reduced the agent to rubber-stamping a heuristic. The current design deliberately avoids this — the queue only controls *when* the agent sees each element, not *what* it should do. The agent receives the same observation and makes the same independent decision regardless of queue position.

Queue Stability Across Mesh Changes

The queue stores **element IDs**, not indices into the active array. This is a critical implementation detail. When the agent refines or coarsens an element, the active array changes: elements are inserted or removed, and the indices of all subsequent elements shift. If the queue stored indices, every action would invalidate the rest of the queue.

By storing IDs, the queue remains valid across mesh changes. At each step, the environment resolves the next ID in the queue to its current index in the active array. If the ID is no longer present (the element was consumed by a cascade or coarsened away), it is simply skipped.

Queue Dynamics Within a Round

The queue is built once at the start of each round and is not rebuilt mid-round. As the agent processes elements:

- **Cascade-consumed elements are skipped.** If a balance cascade refines an element that hasn't been visited yet, that element's ID disappears from the active list. When the queue reaches it, the ID-to-index resolution fails and the element is skipped.
- **Cascade-created children are not added.** New elements created by cascades do not appear in the current round's queue. They will appear in the next round's queue when it is rebuilt.
- **Observations are always fresh.** Although the queue order is fixed, the observation is constructed at the moment of presentation from the current mesh state. Later elements in the queue see the full consequences of all earlier actions and cascades.

Queue Rebuild Between Rounds

Between rounds, the queue is rebuilt from scratch:

1. Error indicators are recomputed on the current (post-action) mesh.
2. Priorities are recalculated using the same fixed thresholds $e_{\max}$, $e_{\min}$.
3. All active elements — including children created by refinement in the previous round — are included in the new queue.
4. The cascade-tracking set (`consumed_elements`) is cleared for the new round.

This is how multi-level refinement works in practice: an element refined from level 0 to level 1 in round 1 produces two children. Those children appear in round 2's queue with their own error indicators and priorities. If they still have high error, the agent can refine them to level 2.

Key Points

1. **The queue is a presentation heuristic, not a decision-maker.** It controls when the agent sees each element, not what the agent should do.
2. **Unified ordering by urgency.** Under-refined and over-refined elements are ranked on the same scale — no separation by action type.
3. **Element IDs provide stability.** The queue survives mesh changes within a round because IDs are persistent even as array indices shift.
4. **Fresh observations despite fixed ordering.** Queue order is set at round start, but each element's observation reflects the true current mesh state at the moment of presentation.